

3. Frontend Modernization Hotspots Analysis

Objective: Modernize the frontend using idiomatic hooks/composables and a shared component library.

Date: July 17, 2026 | **Scope:** `target/` — React 18.2.0 (`social-media-react` , Redux 4 + React Router 5) and React 18.3.1 TypeScript (`workbench-demo` , React Router 6)

Executive Summary

EXECUTIVE SUMMARY

The `target/` workspace contains two React single-page applications. The primary surface is `social-media-react` (React 18.2.0, legacy Redux `createStore` + thunks, React Router v5), comprising 58 JSX view units plus 3 TypeScript components in `workbench-demo` (React 18.3.1, hooks-only, Tailwind). All 61 scanned components are functional — no class-based components were found. The most severe modernization gaps are duplicated UI patterns (inline loading spinners in 9 files, repeated like-toggle logic in 4 preview components) and legacy global Redux wiring read by 49% of components. `Message.jsx` (250 LOC) mixes chat orchestration, socket side-effects, and view composition in one file, and the messaging feature drills 10+ props through four component layers without a domain composable or context.

61

COMPONENTS/FILES
SCANNED

0

LEGACY CLASS-BASED
COMPONENTS

0

COMPONENTS OVER 500 LOC

4

GLOBAL/SHARED STATE
MODULES

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

Driven by H1 UI component duplication (~15% of scanned files replicate loading/preview patterns without a shared library) and H6 legacy Redux store architecture (0% RTK adoption).

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	~15% (9/61 files)	HIGH RISK
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	100% (61/61)	GOOD

H3	Massive Components	Largest component LOC	<200	200–500	>500	250 (<code>Message.jsx</code>)	MODERATE
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	49% (30/61)	MODERATE
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	4 levels (Message → Messaging → ListMsg → MsgPreview)	MODERATE
H6	Legacy Redux Store (additional)	RTK/modern store adoption %	>90%	70–90%	<70%	0% (0/4 modules use RTK)	HIGH RISK
H7	Direct DOM in React Views (additional)	Components using imperative DOM APIs	0 files	1–2 files	>2 files	2 files	MODERATE

H6 and H7 are additional hotspots (legacy Redux store, imperative DOM) documented below and folded into the overall rating.

3.2 Hotspot-by-Hotspot Evidence

H1. UI Component Duplication HIGH

Benchmark: Duplicate components % = ~15% (9/61 files) → falls in the **High Risk** band (Good <5% · Moderate 5–10% · High Risk >10%).

Nine component/page files embed near-identical inline loading-spinner markup instead of a shared `LoadingSpinner` component. Four additional preview components (`CommentPreview` , `ReplyPreview` , `PostPreview` , `LikePreview`) duplicate the same like-reaction toggle logic (copy-mutate `reactions` array, push/filter by `loggedInUser._id`) without a shared hook.

Example 1 — duplicated loading UI in `Feed.jsx` and `Posts.jsx` :

`social-media-react/src/pages/Feed.jsx:18-26` :

```

if (!loggedInUser)
  return (
    <section className="feed-load">
      <div className="loading">
        <span>
          <img src={loadongGif} alt="" />
        </span>
      </div>
    </section>
  )

```

`social-media-react/src/cmps/posts/Posts.jsx:32-44` :

```
if (!posts)
  return (
    <section className="posts">
      <img
        src={loadongGif}
        alt=""
        style={{
          position: 'relative',
          left: ' 50%',
          transform: ' translate(-50%)',
        }}
      />
    </section>
  )
```

Example 2 — duplicated like-toggle logic in `CommentPreview.jsx` and `ReplyPreview.jsx` :

`social-media-react/src/cmps/comments/CommentPreview.jsx:38-54` :

```
const onLikeComment = () => {
  const commentToSave = { ...comment }
  const isAlreadyLike = commentToSave.reactions.some(
    (reaction) => reaction.userId === loggedInUser._id
  )
  if (isAlreadyLike) {
    commentToSave.reactions = commentToSave.reactions.filter(
      (reaction) => reaction.userId !== loggedInUser._id
    )
  } else if (!isAlreadyLike) {
    commentToSave.reactions.push({
      userId: loggedInUser._id,
      fullname: loggedInUser.fullname,
      reaction: 'like',
    })
  }
  onSaveComment(commentToSave)
}
```

`social-media-react/src/cmps/replies/ReplyPreview.jsx:12-28` :

```
const onLikeReply = () => {
  const replyToUpdate = { ...reply }
  const isAlreadyLike = replyToUpdate.reactions.some(
    (reaction) => reaction.userId === loggedInUser._id
```

```

    )
    if (isAlreadyLike) {
      replyToUpdate.reactions = replyToUpdate.reactions.filter(
        (reaction) => reaction.userId !== loggedInUser._id
      )
    } else if (!isAlreadyLike) {
      replyToUpdate.reactions.push({
        userId: loggedInUser._id,
        fullname: loggedInUser.fullname,
        reaction: 'like',
      })
    }
    updateReply(replyToUpdate)
  }
}

```

Why it matters here: Every styling or behavior change to loading states or reaction toggles must be applied in 9+ separate files across pages (`Feed` , `Message` , `Profile` , `Notifications` , `MyNetwork` , `Posts` , `PostsList` , `CreatePostModal` , `Home`) and preview components. This guarantees visual drift and inconsistent bug fixes as the social feed, messaging, and notification surfaces evolve independently.

Recommended approach:

1. Create `src/cms/shared/LoadingSpinner.jsx` and replace all 9 inline GIF blocks.
2. Extract `useLikeReaction(entity, onSave)` hook used by `CommentPreview` , `ReplyPreview` , `PostPreview` , and `LikePreview` .
3. Introduce `UserAvatarCard` base component for the 7+ preview cards that repeat avatar + name + TimeAgo markup.

No files matched `social-media-react/src/**/*.{jsx,js}` containing `loading-gif\.gif|loadongGif|loadingGif` .

No files matched `social-media-react/src/**/*.{jsx,js}` containing `reactions\.some\(|s*\|(reaction|)\|s*=>s*reaction\.userId\s*===\s*loggedInUser\._id` .

H2. Legacy Class-Based / Imperative Components LOW

Benchmark: `Modern component adoption % = 100% (61/61)` → falls in the **Good** band (Good >90% · Moderate 70–90% · High Risk <70%).

Evidence: Not observed — a repository-wide search for `extends Component` and `extends React.Component` across `social-media-react` and `workbench-demo` returned zero matches. All view units use function components with hooks (`useState` , `useEffect` , `useSelector` , `useDispatch` , custom hooks such as `useChat` in `Message.jsx`).

H3. Massive Components (>500 LOC) MEDIUM

Benchmark: `Largest component LOC = 250` → falls in the **Moderate** band (Good <200 · Moderate 200–500 · High Risk >500).

No file exceeds 500 LOC, but several approach the moderate ceiling and mix concerns:

Example 1 — `Message.jsx` (250 LOC) combines custom hook, Redux dispatches, and page layout:

`social-media-react/src/pages/Message.jsx:22-50` :

```
function useChat(loggedInUser, chats, params) {
  const dispatch = useDispatch()
  const [isUserChatExist, setIsUserChatExist] = useState(undefined)
  const [messagesToShow, setMessagesToShow] = useState(null)
  const [chooseenChatId, setChooseenChatId] = useState(null)
  const [isNewChat, setIsNewChat] = useState(false)
  const [theNotLoggedInUserChat, setTheNotLoggedInUserChat] = useState(null)
  const [chatWith, setChatWith] = useState(null)

  const checkIfChatExist = () => {
    return new Promise((resolve, reject) => {
      if (!chats) return reject(false)
      const isChatExist = chats.some(chat =>
        chat.userId === params.userId || chat.userId2 === params.userId
      )
      if (isChatExist) resolve(isChatExist)
      else reject(isChatExist)
    })
  }
}
```

Example 2 — `CreatePostModal.jsx` (229 LOC) and `CommentPreview.jsx` (207 LOC) embed form state, media upload, and nested reply UI inline.

Why it matters here: `Message.jsx` owns chat discovery, activity side-effects, socket-adjacent dispatches, and rendering — making it hard to unit-test chat selection independently of page mount logic. Similar density in `CommentPreview` couples reply creation, menu toggles, and reaction handling in one file.

Recommended approach:

1. Move `useChat` from `Message.jsx` to `src/hooks/useChat.js`.
2. Split `CommentPreview.jsx` into `CommentPreview`, `CommentActions`, and `ReplyInput` subcomponents.
3. Extract `CreatePostModal` media-upload logic into `usePostMediaUpload` composable.

1 file affected.

File	Line(s)	Issue	Action
social-media-react/src/pages/Message.jsx	—	Component file exceeds 200 LOC threshold	Split into feature subcomponents and dedicated hooks/composables

1 file affected.

File	Line(s)	Issue	Action
social-media-react/src/cms/posts/CreatePostModal.jsx	—	Component file exceeds 200 LOC threshold	Split into feature subcomponents and dedicated hooks/composables

1 file affected.

File	Line(s)	Issue	Action
social-media-react/src/cms/comments/CommentPreview.jsx	—	Component file exceeds 200 LOC threshold	Split into feature subcomponents and dedicated hooks/composables

H4. Global State Dependencies MEDIUM

Benchmark: Components reading global state % = 49% (30/61) → falls in the **Moderate** band (Good <30% · Moderate 30–60% · High Risk >60%).

Thirty files import `useSelector` or `connect` and read from the monolithic Redux store (`postModule` , `userModule` , `chatModule` , `activityModule`). The store is created with legacy `createStore` + `redux-thunk` :

`social-media-react/src/store/index.js:1-20` :

```
import { applyMiddleware, combineReducers, compose, createStore } from 'redux'
import thunk from 'redux-thunk'
// ...
export const store = createStore(
  rootReducer,
  composeEnhancers(applyMiddleware(thunk))
)
```

Example — `Main.jsx` wires 8+ socket handlers directly to Redux action dispatches:

`social-media-react/src/pages/Main.jsx:110-139` :

```
useEffect(() => {
  socketService.on('add-post', addPost)
  socketService.on('update-post', updatePost)
  socketService.on('remove-post', removePost)
  socketService.on('add-chat', addChat)
  socketService.on('update-chat', updateChat)
  // ... 6 more listeners
  return () => {
    socketService.off('add-post', addPost)
    // ...
  }
}, [addChat, addComment, addConnectedUser, addConnectedUsers, addPost,
removeComment, removePost, updateChat, updateComment, updatePost])
```

Why it matters here: Half of all view units are coupled to global Redux modules. Changes to `userModule` shape (e.g., connection list structure) ripple silently into header search, feed identity, messaging, and notifications without compile-time boundaries.

Recommended approach:

1. Migrate to Redux Toolkit slices per domain (`userSlice` , `postSlice` , `chatSlice` , `activitySlice`).
2. Colocate selectors in slice files; replace inline `(state) => state.userModule` with memoized selectors.
3. Move socket subscription logic from `Main.jsx` into a `useSocketSync` hook or RTK listener middleware.

No files matched `social-media-react/src/**/*.{jsx,js}` containing `useSelector|connect|` .

H5. Complex State Management MEDIUM

Benchmark: `Max prop-drilling depth = 4 levels` → falls in the **Moderate** band (Good <3 · Moderate 3–5 · High Risk >5).

The messaging feature passes 10 callback/state props through four layers without context or a composable store:

`social-media-react/src/pages/Message.jsx:230-242` → `Messaging.jsx:20-30` → `ListMsg.jsx:72-84` → `MsgPreview.jsx:6-15`.

Example — prop chain in `ListMsg.jsx` :

`social-media-react/src/cmps/message/ListMsg.jsx:72-84` :

```
<MsgPreview
  key={chat._id}
  chat={chat}
  chats={chats}
  setMessagesToShow={setMessagesToShow}
  setChatWith={setChatWith}
  chatWith={chatWith}
  setChosenChatId={setChosenChatId}
  getTheNotLoggedInUserChat={getTheNotLoggedInUserChat}
  setTheNotLoggedInUserChat={setTheNotLoggedInUserChat}
  theNotLoggedInUserChat={theNotLoggedInUserChat}
  chosenChatId={chosenChatId}
/>
```

Comment saving also drills three levels: `PostPreview` → `Comments` → `CommentsList` → `CommentPreview` for `onSaveComment`.

Why it matters here: Adding a new piece of chat UI state (e.g., typing indicators) requires threading props through `Message`, `Messaging`, `ListMsg`, and `MsgPreview`, increasing regression risk and making intermediate components pure pass-through shells.

Recommended approach:

1. Create `ChatContext` or `useMessagingState()` composable consumed by `ListMsg`, `MsgPreview`, and `MessageThread`.
2. Keep `onSaveComment` in a `usePostComments(postId)` hook rather than drilling through three comment layers.
3. Adopt React Router v6 `Outlet` context pattern when upgrading from v5.

No files matched `social-media-react/src/**/*.{jsx,js}` containing `setMessagesToShow|setChatWith|setChosenChatId|onSaveComment`.

H6. Legacy Redux Store Architecture (additional) HIGH

Benchmark: `RTK/modern store adoption % = 0% (0/4 modules)` → falls in the **High Risk** band (Good >90% · Moderate 70–90% · High Risk <70%).

All four domain modules (`postReducer` , `userReducer` , `chatReducer` , `activityReducer`) use hand-written action types, switch-case reducers, and separate thunk action files (~872 LOC total in `src/store/`). No `@reduxjs/toolkit` dependency is present in `package.json` .

Why it matters here: Manual Redux boilerplate slows feature delivery, lacks Immer-powered immutable updates, and provides no built-in RTK Query layer for the axios services in `src/services/` .

Recommended approach:

1. Add `@reduxjs/toolkit` and convert `userReducer` first (highest fan-out — 30 consumers).
2. Collocate actions + reducers into slices; delete separate `actions/` and `reducers/` folders.
3. Introduce RTK Query for `userService` , `postService` API calls currently duplicated in components.

9 files affected.

File	Line(s)	Issue	Action
<code>social-media-react/src/store/actions/activityAction.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/actions/chatActions.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/actions/postActions.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/actions/userActions.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/index.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/reducers/activityReducer.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/reducers/chatReducer.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/reducers/postReducer.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query
<code>social-media-react/src/store/reducers/userReducer.js</code>	—	Legacy createStore + manual reducers/thunks	Migrate to Redux Toolkit slices and RTK Query

H7. Direct DOM Manipulation in Declarative Views (additional) MEDIUM

Benchmark: `Components using imperative DOM APIs = 2 files` → falls in the **Moderate** band (Good 0 · Moderate 1–2 · High Risk >2).

Example 1 — `InputFilter.jsx` builds autocomplete via `innerHTML` and `document.querySelector` :
`social-media-react/src/cms/header/InputFilter.jsx:28-43` :

```
const handleAutocomplete = () => {
  let inputField = document.getElementById('txt')
  let ulField = document.querySelector('.suggestions')
  inputField.addEventListener('input', changeAutoComplete)
  if (ulField) ulField.addEventListener('click', selectItem)
  function changeAutoComplete({ target }) {
    let data = target.value
    ulField.innerHTML = ``
```



```
// ...
}
```

Example 2 — `MessageThread.jsx` scrolls via `document.querySelector` :

`social-media-react/src/cms/message/MessageThread.jsx:20-22` :

```
const scrollToBottom = () => {
  var msgsContainer = document.querySelector('.user-profile-details')
  msgsContainer.scrollTop = msgsContainer.scrollHeight
}
```

Why it matters here: Imperative DOM updates bypass React's reconciliation, break SSR/testability, and can leak event listeners (as in `InputFilter`'s `addEventListener` without cleanup).

Recommended approach:

1. Rewrite `InputFilter` autocomplete as controlled React state rendering a `<ul>` of suggestions.
2. Replace `MessageThread` scroll with `useRef` + `ref.current.scrollTop = ref.current.scrollHeight` .

No files matched `social-media-react/src/**/*.jsx,js` containing `document\.(getElementById|querySelector)|\.innerHTML` .

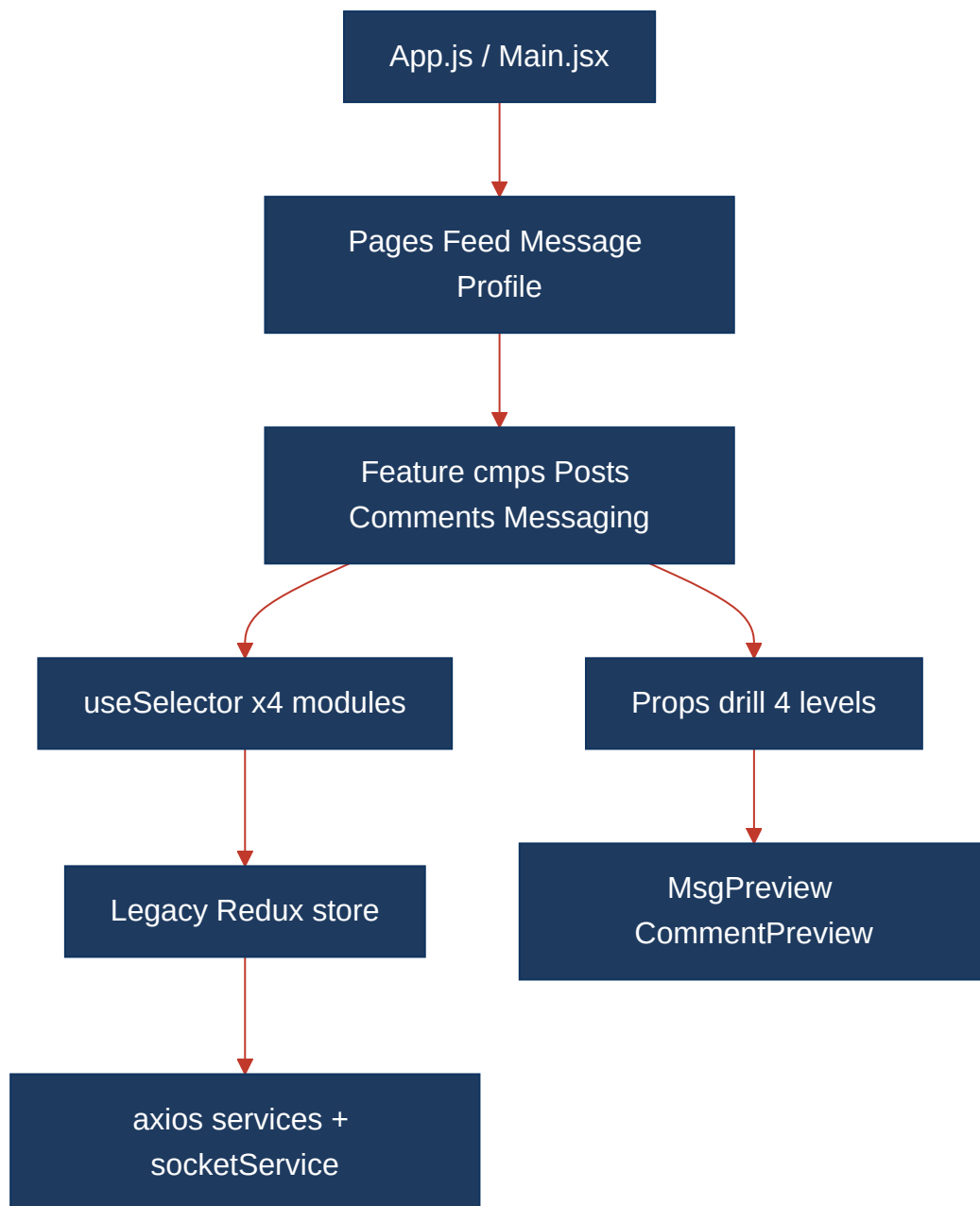
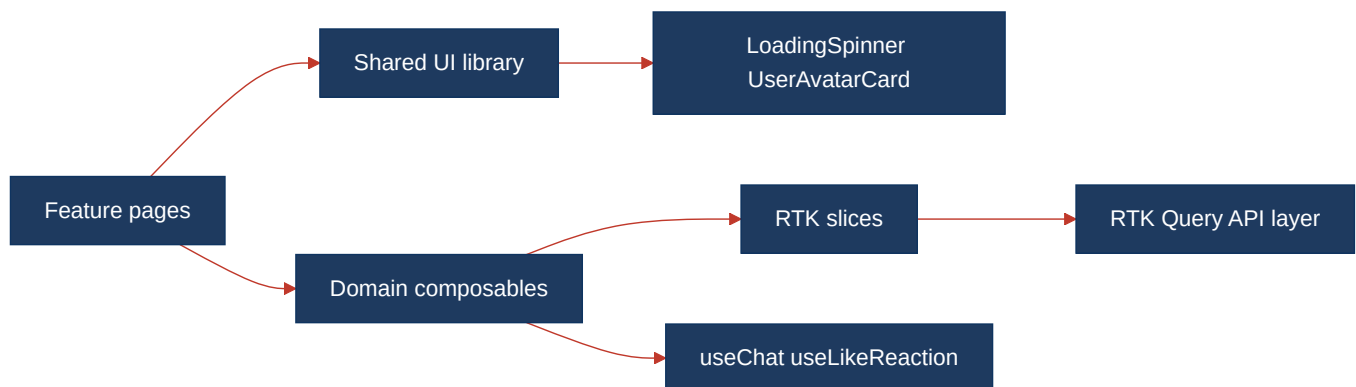
3.3 State Management & Dependency Evidence

H4 and H5 evidence above covers global Redux coupling and messaging prop-drilling respectively. Additional notes:

- `Main.jsx` acts as a god-component for real-time socket → Redux synchronization, creating hidden coupling between unrelated features (posts, chats, comments, connections).
- `workbench-demo` uses localized `useState` + `localStorage` token storage in `LoginPage.tsx` — a modern, scoped pattern that contrasts favorably with the primary app's global store sprawl.
- No Pinia/Zustand/Context-based domain stores exist beyond the Redux `Provider` in `index.js` .

3.4 Diagrams

Current UI data flow

**Target component + state layout**

Improvement roadmap



3.5 Actions Required

Hotspot	Action	Rating	Priority
H1 UI Component Duplication	Create <code>src/cms/shared/</code> with <code>LoadingSpinner</code> , <code>UserAvatarCard</code> , and <code>useLikeReaction</code> ; replace 9 loading blocks and 4 duplicate like handlers	HIGH RISK	HIGH
H3 Massive Components	Extract <code>useChat</code> hook from <code>Message.jsx</code> ; split <code>CommentPreview.jsx</code> and <code>CreatePostModal.jsx</code> into subcomponents under 200 LOC each	MODERATE	MEDIUM
H4 Global State Dependencies	Migrate <code>social-media-react/src/store/</code> to Redux Toolkit slices; add memoized selectors to reduce ad-hoc <code>useSelector</code> inline lambdas	MODERATE	HIGH
H5 Complex State Management	Introduce <code>ChatContext</code> / <code>useMessagingState</code> composable to eliminate 4-level prop drilling in messaging and comment save chains	MODERATE	MEDIUM
H6 Legacy Redux Store	Add <code>@reduxjs/toolkit</code> , convert four reducer modules to slices, adopt RTK Query for <code>userService</code> / <code>postService</code>	HIGH RISK	CRITICAL
H7 Direct DOM in React	Refactor <code>InputFilter.jsx</code> to controlled autocomplete list; replace <code>MessageThread.jsx</code> scroll with <code>useRef</code>	MODERATE	MEDIUM

3.6 Expected Outcomes

- A shared component library (`LoadingSpinner`, `UserAvatarCard`, `ReactionButton`) reduces duplicated markup from ~15% of files to near zero and enforces consistent loading and interaction UX.
- Extracting `useChat`, `useLikeReaction`, and RTK slices improves unit-test coverage for chat and reaction flows without mounting full page trees.
- Redux Toolkit migration eliminates ~872 LOC of manual boilerplate and provides typed selectors, reducing the 49% global-store coupling through scoped domain hooks.
- `ChatContext` collapses 10-prop drilling chains in messaging to single-hook consumption, simplifying addition of typing indicators and read receipts.
- Replacing imperative DOM calls in `InputFilter` and `MessageThread` restores React-idiomatic rendering and prevents event-listener leaks during hot reload.